

HỆ THỐNG PHÁT HIỆN HƯ HỎNG MẶT ĐƯỜNG

Cải tiến kiến trúc YOLO11s-P2 (ECA · Mish · RMSA) và
Triển khai ứng dụng Web nhận diện thời gian thực

*Bài toán: Phát hiện & phân loại 3 lớp hư hỏng mặt đường
(Pothole – Ổ gà · Crack – Vết nứt · Manhole – Nắp cống)*

Kiến trúc nền: YOLO11s-P2 | Framework: Ultralytics · PyTorch · FastAPI
Môi trường thực nghiệm: NVIDIA GeForce RTX 3060 Laptop GPU (6GB VRAM)

Nguyễn Văn Tú · Bùi Ngọc Bảo Trân · Nguyễn Anh Việt

Mục lục

1. Giới thiệu
 2. Cơ sở lý thuyết và kiến trúc cải tiến
 3. Phương pháp thực nghiệm
 4. Kết quả thực nghiệm và Benchmark
 5. Triển khai hệ thống Web Application
 6. Kết luận và hướng phát triển
- Phụ lục — Giải đáp câu hỏi kỹ thuật

1. Giới thiệu:

1.1. Bối cảnh và động lực:

Hư hỏng mặt đường — bao gồm ổ gà (pothole), vết nứt (crack) và tình trạng nắp cống (manhole) không đạt tiêu chuẩn — là nguyên nhân trực tiếp gây mất an toàn giao thông và làm gia tăng chi phí bảo trì hạ tầng. Phương pháp khảo sát thủ công truyền thống tốn nhiều nhân lực, thời gian và mang tính chủ quan cao. Sự phát triển của các mô hình phát hiện đối tượng (object detection) dựa trên học sâu, đặc biệt là họ mô hình YOLO (You Only Look Once), đã mở ra khả năng tự động hoá quá trình khảo sát này với tốc độ xử lý thời gian thực.

Dự án hướng đến hai mục tiêu song song:

- (i) Nghiên cứu và đánh giá thực nghiệm các phương án cải tiến kiến trúc YOLO11s-P2 nhằm cải thiện sự cân bằng giữa độ chính xác và tốc độ suy luận trong bài toán phát hiện hư hỏng mặt đường.
- (ii) Triển khai mô hình tốt nhất thành một ứng dụng web hoàn chỉnh, cho phép người dùng tải ảnh lên và nhận kết quả phát hiện trực quan.

1.2. Mục tiêu nghiên cứu:

- Xây dựng một quy trình thực nghiệm công bằng (cùng dữ liệu, cùng seed, cùng cấu hình huấn luyện) để so sánh khách quan 5 phương án kiến trúc: Baseline, YOLO11s-P2 + ECA, YOLO11s-P2 + Mish, YOLO11s-P2 + ECA + Mish, YOLO11s-P2 + RMSA.
- Đánh giá từng phương án trên hai trục: độ chính xác phát hiện (mAP50, mAP50-95, Precision, Recall) và hiệu năng suy luận (latency, FPS, số tham số, GFLOPs).
- Thực hiện một vòng benchmark tốc độ suy luận nghiêm ngặt, có kiểm soát nhiễu đo lường, để xác nhận lại các kết luận sơ bộ.
- Đóng gói mô hình tốt nhất thành dịch vụ suy luận (inference service) qua REST API và xây dựng giao diện web cho phép tương tác trực tiếp.

1.3. Phạm vi bài toán:

Bài toán được giới hạn ở phát hiện và phân loại 3 lớp đối tượng trên ảnh chụp mặt đường: Pothole (ổ gà), Crack (vết nứt) và Manhole (nắp cống). Đầu vào là ảnh tĩnh (JPEG/PNG), đầu ra là các bounding box kèm nhãn lớp và độ tin cậy (confidence). Toàn bộ 5 mô hình sử dụng chung một kiến trúc nền YOLO11s-P2 với đầu ra bổ sung ở tầng P2 (stride 4) nhằm cải thiện khả năng phát hiện vật thể nhỏ và mảnh — đặc điểm điển hình của vết nứt và ổ gà.

2. Cơ sở lý thuyết và kiến trúc cải tiến:

2.1. Kiến trúc nền — YOLO11s-P2:

YOLO11 là thế hệ kiến trúc one-stage detector mới của Ultralytics, cải tiến backbone/neck (khối C3k2, C2PSA) so với các thế hệ trước. Biến thể "s" (small) được lựa chọn vì cân bằng giữa tốc độ suy luận và khả năng học đặc trưng, phù hợp triển khai thực tế trên phần cứng vừa phải. Hậu tố "P2" chỉ việc bổ sung một đầu ra phát hiện ở tầng đặc trưng có độ phân giải cao hơn (stride 4, so với stride 8 của P3 trong YOLO chuẩn) — giúp mô hình giữ lại nhiều chi tiết không gian hơn cho các vật thể nhỏ, mảnh và dài như vết nứt mặt đường.

Toàn bộ 5 phương án thực nghiệm trong báo cáo đều dùng chung một cấu hình kiến trúc nền và cùng bộ trọng số pretrained cho phần backbone, chỉ khác nhau ở các thành phần được chèn thêm vào phần neck/head — nhằm cô lập chính xác tác động của từng cải tiến.

2.2. ECA — Efficient Channel Attention:

ECA là cơ chế attention theo kênh (channel attention), được thiết kế nhẹ hơn nhiều so với SE-Net (Squeeze-and-Excitation). Thay vì dùng hai lớp fully-connected để học tương tác giữa các kênh, ECA sử dụng Global Average Pooling để nén mỗi kênh đặc trưng thành một giá trị vô hướng, sau đó áp dụng một phép tích chập 1 chiều (Conv1D, kích thước nhân k thích ứng theo số kênh, không có bias) để học tương tác cục bộ giữa các kênh lân cận, rồi đưa qua hàm Sigmoid để tạo trọng số kênh, cuối cùng nhân trọng số này trở lại vào feature map gốc.

Trong thực nghiệm, module ECA được chèn ngay sau các khối C3k2 tại layer 16, 19 và 22 của neck — là các tầng xử lý đặc trưng liên quan trực tiếp đến nhánh P2/P3, nơi tập trung xử lý các vật thể nhỏ.

2.3. Mish — Hàm kích hoạt:

Mish là hàm kích hoạt phi tuyến được định nghĩa bởi công thức $f(x) = x \cdot \tanh(\text{softplus}(x))$, trong đó $\text{softplus}(x) = \ln(1 + e^x)$. So với SiLU ($f(x) = x \cdot \text{sigmoid}(x)$) — hàm kích hoạt mặc định của YOLO11 — Mish có đặc tính mượt hơn, không đơn điệu (non-monotonic) tại vùng giá trị âm gần 0, và không có điểm "chết" hoàn toàn, giúp gradient lan truyền ổn định hơn qua các lớp sâu.

Trong thực nghiệm, SiLU được thay bằng Mish tại toàn bộ neck và head, trong khi backbone vẫn giữ nguyên SiLU để tương thích với trọng số pretrained trên COCO.

2.4. ECA + Mish — Ablation kết hợp:

Phương án này áp dụng đồng thời cả ECA (tại layer 16, 19, 22) và Mish (toàn bộ neck/head) trên cùng một mô hình, nhằm kiểm tra giả thuyết liệu hai cải tiến có "cộng hưởng" (tác động tích lũy tích cực) khi kết hợp hay không.

2.5. RMSA — Residual Multi-Scale Attention:

RMSA là module tự thiết kế, phức tạp hơn ECA, thay thế trực tiếp hai khối C3k2 tại layer 16 và 19. Cấu trúc gồm: một nhánh depthwise convolution 3×3 (bắt đặc trưng cục bộ, chi tiết nhỏ), song song với một nhánh depthwise convolution 5×5 (receptive field rộng hơn, phù hợp với các vết nứt dài); hai nhánh được nối (concatenate) rồi hợp nhất qua một phép tích chập 1×1 ; kết quả hợp nhất tiếp tục được đưa qua một module ECA lồng bên trong để lọc kênh; sau đó một phép tích chập 1×1 khác đưa số kênh về đúng kích thước ban đầu; cuối cùng một kết nối tắt (residual/skip connection) cộng trực tiếp với đầu vào ban đầu để ổn định quá trình huấn luyện.

Ghi chú: RMSA là module phức tạp nhất trong 4 phương án cải tiến, được thiết kế với kỳ vọng khai thác đặc trưng đa tỉ lệ tốt hơn, nhưng — như trình bày ở Mục 4 — kết quả thực nghiệm cho thấy độ phức tạp bổ sung này không chuyển hoá thành cải thiện độ chính xác trên tập dữ liệu hiện tại.

3. Phương pháp thực nghiệm:

3.1. Dữ liệu:

Bộ dữ liệu gồm tổng cộng 2.009 ảnh mặt đường, được gán nhãn cho 3 lớp: Pothole, Crack, Manhole. Dữ liệu được chia theo tỉ lệ xấp xỉ 51 : 20 : 29 (train : val : test):

Tập dữ liệu	Số lượng ảnh	Tỉ lệ
Train	1.029	≈ 51,2%
Validation	401	≈ 20,0%
Test	579	≈ 28,8%
Tổng	2.009	100%

Bảng 1. Phân chia tập dữ liệu train/val/test.

3.2. Cấu hình huấn luyện dùng chung:

Cả 5 mô hình được huấn luyện với cùng seed và siêu tham số nhằm đảm bảo tính công bằng khi so sánh:

Tham số	Giá trị
Kích thước ảnh đầu vào (imgsz)	640 × 640
Số epoch tối đa	100
Early-stopping patience	20
Batch size	8
Optimizer	auto (Ultralytics)
Learning rate (lr0 / lrf)	0,01 / 0,01
Momentum / Weight decay	0,937 / 0,0005
Mixed precision (AMP)	Bật (khi có GPU)
Seed	42 (cố định toàn bộ random, numpy, torch)
Pretrained backbone	Có (COCO)

Bảng 2. Cấu hình huấn luyện chung cho cả 5 phương án.

3.3. Quy trình đánh giá và benchmark:

Quy trình đánh giá được thực hiện theo hai giai đoạn. Giai đoạn thứ nhất diễn ra ngay trong từng notebook huấn luyện: sau khi huấn luyện xong, mỗi mô hình được đánh giá trên tập validation (mAP50, mAP50-95, Precision, Recall) và benchmark latency sơ bộ bằng hàm benchmark_model() (20 lần chạy khởi động, 100 lần đo, đo bằng time.perf_counter() có đồng bộ CUDA). Do mỗi notebook được chạy ở một phiên (session) huấn luyện riêng biệt, các phép đo latency sơ bộ này có thể chịu ảnh hưởng bởi điều kiện phần cứng không đồng nhất giữa các phiên (nhiệt độ GPU, trạng thái bộ nhớ đệm CUDA, tải hệ thống nền).

Để khắc phục hạn chế trên, giai đoạn thứ hai bổ sung một vòng benchmark nghiêm ngặt (rigorous benchmark), trong đó cả 5 mô hình được nạp cùng lúc trong một phiên chạy duy nhất, thứ tự đo được xáo trộn ngẫu nhiên qua 5 vòng lặp, cudnn.benchmark được tắt để loại bỏ nhiễu do tự động tinh chỉnh kernel, và số lượt đo được tăng lên 1.000 lượt suy luận thuần GPU (pure forward pass) cho mỗi mô hình ở cả hai độ chính xác FP32 và FP16, cùng với 100 lượt đo pipeline đầy đủ (end-to-end: tiền xử lý → suy luận → hậu xử lý NMS) trên ảnh validation thật.

4. Kết quả thực nghiệm và Benchmark:

4.1. Bảng so sánh tổng hợp độ chính xác:

Mô hình	mAP50	mAP50-95	Precision	Recall	Tham số
Baseline	0,3923	0,1693	0,4491	0,4095	9.575.292
+ ECA	0,3851	0,1638	0,4420	0,4259	9.575.292
+ Mish 	0,3880	0,1698	0,4489	0,4143	9.575.292
+ ECA + Mish	0,3668	0,1579	0,4154	0,4290	9.575.292
+ RMSA	0,3681	0,1546	0,4107	0,4160	9.575.292

Bảng 3. So sánh độ chính xác của 5 phương án kiến trúc trên tập validation.

Kết quả cho thấy Mish là phương án duy nhất vượt qua baseline về mAP50-95 (0,1698 so với 0,1693, tăng +0,0005), đồng thời cũng đạt mAP50 cao nhất trong nhóm cải tiến (0,3880). ECA và RMSA khi dùng riêng lẻ đều làm giảm nhẹ độ chính xác so với baseline. Đáng chú ý, kết hợp ECA+Mish cho kết quả thấp hơn cả khi dùng từng module riêng lẻ, cho thấy hai cải tiến không cộng hưởng trong cấu hình huấn luyện này.

4.2. Benchmark tốc độ suy luận nghiêm ngặt:

Bảng dưới trình bày kết quả benchmark suy luận thuần GPU (FP32), đo trên 1.000 lượt forward pass cho mỗi mô hình, thứ tự xáo trộn ngẫu nhiên qua 5 vòng để loại bỏ thiên lệch do trôi nhiệt/xung nhịp:

Xếp hạng	Mô hình	Median (ms)	Mean (ms)	Std	FPS
1	ECA	21,50	22,76	4,66	46,5
2	Baseline	21,52	23,07	4,84	46,5
3	Mish	21,57	23,74	7,13	46,4
4	ECA + Mish	21,75	23,69	7,73	46,0
5	RMSA	22,27	23,91	5,27	44,9

Bảng 4. Benchmark suy luận thuần GPU FP32 ($N = 1.000$ lượt/mô hình).

Bảng tiếp theo trình bày kết quả đo pipeline đầy đủ (end-to-end), gồm tiền xử lý, suy luận và hậu xử lý NMS, trên 100 ảnh validation thật:

Mô hình	Median (ms)	Mean (ms)	Std	FPS
Baseline	27,78	28,85	5,13	36,0
ECA	22,59	22,71	1,76	44,3
Mish 	21,94	22,09	1,04	45,6
ECA + Mish	22,64	23,23	2,68	44,2
RMSA	23,01	23,79	2,44	43,5

Bảng 5. Benchmark end-to-end pipeline ($N = 100$ ảnh validation thật).

4.3. Đánh đổi giữa tốc độ và độ chính xác (Trade-off):

Tổng hợp cả hai trục đánh giá cho thấy: (1) khác biệt latency thuần GPU giữa 5 biến thể là rất nhỏ (21,50–22,27 ms, chênh lệch dưới 4%), nằm trong phạm vi nhiễu đo lường bình thường của một GPU laptop; (2) toàn bộ 5 mô hình có số tham số và GFLOPs giống hệt nhau (9.575.292 tham số, $\approx 28,95$ GFLOPs) — do ECA và Mish gần như không làm thay đổi kích thước mô hình (xem giải thích chi tiết ở Phụ lục); (3) trên phép đo end-to-end thực tế, Mish là mô hình nhanh nhất (21,94 ms, 45,6 FPS) và đồng thời đạt độ chính xác cao nhất trong nhóm cải tiến — do đó được lựa chọn là phương án triển khai chính thức cho ứng dụng web.

4.4. Thảo luận:

- Mish (YOLO11s-P2 + Mish) là lựa chọn tối ưu tổng thể: không tăng tham số, cải thiện nhẹ độ chính xác, và nhanh nhất trên pipeline thực tế.
- ECA giúp giảm rõ rệt latency so với baseline trên phép đo end-to-end (22,59 ms so với 27,78 ms) nhưng đánh đổi bằng độ chính xác thấp hơn baseline.
- Kết hợp ECA+Mish không mang lại hiệu ứng cộng hưởng — hai module có thể xung đột trong cùng ngân sách huấn luyện (100 epoch).
- RMSA — dù có thiết kế phức tạp và kỳ vọng cao — không cải thiện được độ chính xác so với baseline trên bộ dữ liệu và cấu hình huấn luyện hiện tại, có thể do thiếu dữ liệu để tận dụng hết dung lượng biểu diễn bổ sung của module.

5. Triển khai hệ thống Web Application:

Dựa trên kết luận thực nghiệm ở Mục 4, mô hình YOLO11s-P2 + Mish được lựa chọn để đóng gói thành một dịch vụ suy luận hoàn chỉnh, phục vụ qua giao diện web "RoadVision — Road Damage Detection". Hệ thống được thiết kế theo kiến trúc client–server tách biệt hoàn toàn giữa phần suy luận AI (backend) và phần tương tác người dùng (frontend).

5.1. Kiến trúc tổng thể:

Luồng xử lý tổng quát của hệ thống gồm 4 bước chính:

- Người dùng chọn ảnh mặt đường và bấm nút "Nhận diện" trên giao diện web.
- Trình duyệt đóng gói ảnh vào một multipart/form-data request và gửi tới endpoint POST /predict của backend FastAPI.
- Backend giải mã ảnh, chạy suy luận bằng mô hình YOLO11s-P2 + Mish (best.pt), vẽ bounding box lên ảnh kết quả, mã hoá ảnh sang Base64 và trả về một JSON tổng hợp gồm ảnh kết quả cùng danh sách các đối tượng phát hiện được.
- Frontend nhận JSON, hiển thị ảnh kết quả, danh sách chi tiết từng phát hiện (lớp, độ tin cậy) và một bảng tổng hợp số lượng theo từng lớp.

5.2. Backend — dịch vụ suy luận FastAPI (main.py):

Backend được xây dựng bằng FastAPI, expose hai endpoint chính: GET / trả về thông tin trạng thái hệ thống (tên mô hình, thiết bị đang chạy, danh sách lớp, ngưỡng tin cậy) và POST /predict thực hiện suy luận trên ảnh do người dùng gửi lên.

Một điểm kỹ thuật quan trọng cần lưu ý: vì mô hình được huấn luyện với hàm kích hoạt Mish tùy chỉnh (không có sẵn trong thư viện Ultralytics gốc), lớp Mish phải được định nghĩa và đăng ký thủ công vào module tasks của Ultralytics (tasks_module.Mish = Mish) trước khi checkpoint best.pt được nạp — nếu không, quá trình deserialize mô hình (torch.load) sẽ thất bại vì không tìm thấy định nghĩa lớp tương ứng đã được lưu trong checkpoint.

Thành phần	Vai trò
FastAPI + CORSMiddleware	Expose REST API, cho phép gọi từ mọi origin (allow_origins="*") phục vụ phát triển/demo
YOLO(MODEL_PATH)	Nạp checkpoint best.pt (mô hình YOLO11s-P2 + Mish đã huấn luyện)
cv2.imdecode	Giải mã bytes ảnh tải lên thành ma trận ảnh BGR
model.predict(imgsz=640, conf=0.25)	Chạy suy luận với cấu hình giống hết lúc huấn luyện/đánh giá
result.plot()	Vẽ bounding box, nhãn lớp và độ tin cậy lên ảnh gốc
base64.b64encode	Mã hoá ảnh kết quả JPEG để nhúng trực tiếp vào JSON (data URL)

Bảng 6. Các thành phần kỹ thuật chính trong backend.

Endpoint /predict được thiết kế phòng thủ: kiểm tra content-type phải là ảnh, kiểm tra ảnh không rỗng, kiểm tra cv2.imdecode giải mã thành công, và bọc toàn bộ bước suy luận trong khối try/except để trả về mã lỗi HTTP phù hợp (400 cho lỗi đầu vào, 500 cho lỗi xử lý) thay vì để server crash.

5.3. Frontend — giao diện tương tác (index.html + script.js):

Giao diện được xây dựng bằng HTML kết hợp Tailwind CSS, với tiêu đề "Phát hiện và phân loại hư hỏng mặt đường bằng mô hình YOLO11s-P2 + Mish", theo đúng kết luận lựa chọn mô hình ở Mục 4. Giao diện gồm khu vực tải ảnh lên (upload zone), khu vực hiển thị ảnh kết quả đã được vẽ bounding box, khu vực tổng hợp số lượng đối tượng phát hiện (summary) và danh sách chi tiết từng phát hiện kèm độ tin cậy.

Toàn bộ logic tương tác được xử lý trong hàm bất đồng bộ uploadImage() (script.js): đóng gói ảnh vào FormData, gửi request POST tới API_URL (http://localhost:8000/predict) bằng fetch(), quản lý các trạng thái giao diện (đang tải, thành công, lỗi) thông qua việc thêm/xoá lớp CSS "hidden", và cuối cùng render động danh sách các phần tử phát hiện bằng cách tạo các phần tử DOM (createElement) cho từng detection trả về từ backend.

Xử lý lỗi được thực hiện bằng khối try/catch/finally: nếu response.ok = false hoặc data.success = false, một Error được ném ra và bắt lại ở catch để hiển thị thông báo lỗi cho người dùng (alert), trong khi khối finally đảm bảo trạng thái nút bấm và loading indicator luôn được khôi phục về trạng thái ban đầu bất kể kết quả thành công hay thất bại.

5.4. Luồng dữ liệu end-to-end:

Tổng hợp lại, một lượt sử dụng diễn hình đi qua chuỗi xử lý sau: Ảnh (File) → FormData → HTTP POST /predict → cv2.imdecode → YOLO.predict() (imgsz=640, conf=0,25) → trích xuất box/class/confidence → result.plot() vẽ bounding box → mã hoá JPEG → Base64 → JSON response → cập nhật DOM (ảnh kết quả, bảng tổng hợp, danh sách chi tiết). Việc tách rời rõ ràng giữa tầng suy luận (Python/FastAPI) và tầng hiển thị (JavaScript thuần, không dùng framework) giúp hệ thống dễ bảo trì, dễ thay thế mô hình trong tương lai mà không cần chỉnh sửa giao diện.

6. Kết luận và hướng phát triển:

6.1. Kết luận:

Qua quá trình thực nghiệm có kiểm soát trên 5 phương án kiến trúc, YOLO11s-P2 + Mish được xác định là phương án tối ưu nhất cho bài toán phát hiện hư hỏng mặt đường trong phạm vi dự án: không làm tăng số lượng tham số hay độ phức tạp tính toán, cải thiện nhẹ độ chính xác so với baseline (mAP50-95: 0,1693 → 0,1698), và cho tốc độ suy luận nhanh nhất trên pipeline thực tế (21,94 ms/ảnh, 45,6 FPS). Mô hình này đã được triển khai thành công thành một dịch vụ web hoàn chỉnh, cho phép nhận diện hư hỏng mặt đường theo thời gian thực từ ảnh do người dùng tải lên.

6.2. Hạn chế:

- Bộ dữ liệu có quy mô vừa phải (2.009 ảnh), có thể chưa đủ lớn để các module attention phức tạp như RMSA phát huy hết ưu thế lý thuyết.
- Benchmark tốc độ chịu ảnh hưởng của phần cứng cụ thể (RTX 3060 Laptop, 6GB VRAM) và có thể cho kết quả khác trên các cấu hình phần cứng khác (ví dụ GPU dùng TensorRT/cuDNN graph compilation).
- Ứng dụng web hiện tại cấu hình CORS mở (allow_origins="*") và chạy ở môi trường phát triển (localhost), cần bổ sung các biện pháp bảo mật và cấu hình phù hợp trước khi triển khai production.

6.3. Hướng phát triển:

- Mở rộng bộ dữ liệu và áp dụng các kỹ thuật tăng cường dữ liệu (data augmentation) nâng cao để đánh giá lại tiềm năng của RMSA.
- Thử nghiệm lượng tử hoá mô hình (quantization) và biên dịch TensorRT nhằm khai thác tốt hơn phần cứng GPU cho suy luận FP16.
- Bổ sung xác thực (authentication) và giới hạn tốc độ gọi API (rate limiting) cho backend trước khi triển khai thực tế.
- Xây dựng pipeline theo dõi liên tục (continuous monitoring) để phát hiện suy giảm hiệu năng mô hình theo thời gian (model drift) khi áp dụng trên dữ liệu thực tế đa dạng hơn.

Phụ lục — Giải đáp câu hỏi kỹ thuật

Câu hỏi 1: Vì sao khi chèn ECA và Mish vào mô hình, số lượng tham số lại không thay đổi ?

Câu trả lời nằm ở bản chất toán học khác nhau của hai thành phần được chèn thêm.

a) Mish — hoàn toàn không thêm tham số:

Mish là một hàm kích hoạt (activation function) thuần túy — một công thức toán học cố định $f(x) = x \cdot \tanh(\text{softplus}(x))$ áp dụng theo từng phần tử (element-wise), không chứa bất kỳ trọng số (weight) hay bias nào có thể học được. Đây là điểm chung với SiLU ($f(x) = x \cdot \text{sigmoid}(x)$) — hàm kích hoạt gốc mà Mish thay thế. Vì cả hai đều là hàm phi tham số (parameter-free), việc "đổi" SiLU sang Mish thực chất chỉ là thay đổi công thức tính toán được áp dụng lên feature map tại forward pass, không hề chèn thêm bất kỳ layer có trọng số nào vào kiến trúc. Do đó tổng số tham số của mô hình giữ nguyên tuyệt đối 9.575.292, không sai lệch dù chỉ một đơn vị.

b) ECA — gần như không thêm tham số (chứ không phải tuyệt đối bằng 0):

ECA có thêm một lượng tham số cực nhỏ, đến từ một phép tích chập 1 chiều duy nhất: `nn.Conv1d(1, 1, kernel_size=k, bias=False)`. Vì số kênh (`in_channels`) và số bộ lọc (`out_channels`) của phép tích chập này đều bằng 1, số tham số của mỗi module ECA chỉ đúng bằng kích thước nhân k (thường là 3, được tính thích ứng theo công thức $k = \lceil \log_2(C)/2 + 0,5 \rceil$ làm tròn lên số lẻ gần nhất, tối thiểu bằng 3) — và hoàn toàn không có bias. Với 3 vị trí chèn ECA (layer 16, 19, 22), tổng số tham số mới được thêm vào chỉ khoảng 9 tham số — một con số nhỏ hơn 0,0001% so với tổng 9.575.292 tham số của toàn mô hình.

Đây chính là lý do module có tên gọi "Efficient" Channel Attention: khác với SE-Net (Squeeze-and-Excitation) — vốn dùng hai lớp fully-connected để học tương tác giữa các kênh và do đó tốn một lượng tham số tỉ lệ thuận với bình phương số kênh — ECA cố tình thay thế bằng một phép tích chập 1D cực nhỏ, giữ nguyên gần như toàn bộ hiệu quả của attention theo kênh nhưng với chi phí tham số gần như không đáng kể. Khi làm tròn và báo cáo ở độ phân giải tổng số tham số (đơn vị người), sự chênh lệch 9 tham số này biến mất hoàn toàn trong con số 9.575.292 — đó là lý do bảng so sánh tổng hợp (Bảng 3) hiển thị con số tham số giống hệt nhau cho cả 5 mô hình, mặc dù về mặt kỹ thuật ECA có phát sinh thêm (một lượng cực nhỏ) tham số còn Mish thì hoàn toàn không.

Câu hỏi 2: Vì sao khi chèn Mish (có thêm phép tính), latency lại giảm ?

a) Về mặt lý thuyết, Mish tốn nhiều phép tính hơn SiLU, không ít hơn

SiLU chỉ cần tính một hàm sigmoid rồi nhân với đầu vào: $f(x) = x \cdot \text{sigmoid}(x)$. Mish cần tính softplus (bản thân softplus đã chứa một phép lũy thừa và một phép logarit: $\ln(1+e^x)$), sau đó tính thêm một hàm tanh, rồi mới nhân với đầu vào: $f(x) = x \cdot \tanh(\text{softplus}(x))$. Vì vậy, xét theo số phép toán dấu phẩy động (FLOPs) trên mỗi phần tử, Mish đắt hơn SiLU, không rẻ hơn. Nếu tốc độ suy luận chỉ phụ thuộc vào khối lượng phép tính của activation, ta sẽ kỳ vọng Mish chậm hơn hoặc bằng SiLU — chứ không phải nhanh hơn. Do đó, giả thuyết "phép nhân ma trận tự triệt tiêu làm giảm khối lượng tính toán" không đứng vững — về bản chất, thay SiLU bằng Mish không hề loại bỏ hay đơn giản hoá bất kỳ phép tích chập (convolution) nào trong mạng; hai phương án chỉ khác nhau ở công thức của các hàm kích hoạt xen giữa các lớp tích chập, còn toàn bộ số lượng và kích thước các phép tích chập (nơi tiêu tốn phần lớn FLOPs của mạng) là hoàn toàn giống hệt nhau giữa Baseline và Mish.

b) Nguồn gốc thực sự: sự khác biệt latency giữa các phiên benchmark là nhiễu đo lường, không phải hiệu ứng thuật toán

Bằng chứng rõ ràng nhất nằm ngay trong dữ liệu benchmark của đồ án. Ở vòng đo latency sơ bộ (được thực hiện riêng lẻ trong từng notebook huấn luyện, mỗi mô hình một phiên chạy độc lập), Baseline đo được 34,4 ms trong khi Mish chỉ 21,4 ms — chênh lệch rất lớn (khoảng 38%). Tuy nhiên, khi thực hiện lại phép đo trong một vòng benchmark nghiêm ngặt hơn — nạp cả 5 mô hình cùng một phiên, đo xen kẽ ngẫu nhiên qua 5 vòng, tắt cudnn.benchmark để loại bỏ nhiễu tự động tinh chỉnh kernel, và tăng số lượt đo lên 1.000 lượt mỗi mô hình (Bảng 4) — khoảng cách giữa các mô hình gần như biến mất: Baseline đạt 21,52 ms, còn Mish đạt 21,57 ms, chênh lệch chưa tới 0,3%, và độ lệch chuẩn của Mish (7,13 ms) thậm chí còn cao hơn — nghĩa là hai mô hình về cơ bản không thể phân biệt được về tốc độ suy luận thuần GPU khi được đo trong cùng điều kiện.

Điều này cho thấy con số 34,4 ms ban đầu của Baseline nhiều khả năng bị ảnh hưởng bởi các yếu tố ngoài thuật toán: mỗi notebook được chạy ở một phiên huấn luyện riêng biệt, có thể tại thời điểm và trạng thái phần cứng khác nhau (nhiệt độ GPU sau nhiều giờ huấn luyện, trạng thái bộ nhớ đệm/cấp phát CUDA lần đầu của phiên, tải hệ thống nền của máy tính laptop tại thời điểm đo, hoặc việc cudnn tự động tinh chỉnh kernel — cudnn.benchmark — chưa được kiểm soát chặt trong các notebook huấn luyện riêng lẻ). Đây chính là lý do nhóm xây dựng thêm notebook benchmark nghiêm ngặt (07_benchmark_all_models.ipynb): để loại trừ các yếu tố nhiễu nói trên và đo lại một cách công bằng, có kiểm soát.

Kết luận nhất quán rút ra từ vòng benchmark nghiêm ngặt là: khác biệt latency thuần GPU giữa toàn bộ 5 biến thể kiến trúc là rất nhỏ (chênh lệch dưới 4%, từ 21,50 đến 22,27 ms) và nằm trong phạm vi nhiễu đo lường bình thường của một GPU laptop, không phản ánh một lợi thế tốc độ có ý nghĩa thống kê của riêng Mish so với các phương án khác ở cấp độ suy luận thuần GPU. Lợi thế tốc độ thực sự và ổn định của Mish chỉ thể hiện rõ ở phép đo end-to-end trên ảnh thật (Bảng 5, 21,94 ms — nhanh nhất trong 5 mô hình), nơi độ lệch chuẩn của Mish cũng thấp nhất (1,04 ms) — cho thấy pipeline của Mish không chỉ nhanh mà còn ổn định hơn, nhiều khả năng nhờ vào đặc tính mượt (smooth) của hàm kích hoạt giúp quá trình numeric bên trong pipeline suy luận (bao gồm cả bước decode box và NMS) ít biến động hơn, chứ không phải vì bản thân phép tính activation rẻ hơn.